Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 7 – Override and Specialise Behaviour 4 Marks

Bloom's Level: K4 – Analyse

Identify the behaviour that must differ between subclasses and implement it by method overriding.

Discuss:

- Which methods are overridden (for example: calculateFee()) and why the subclass version is needed
- Use of the @Override annotation and the rules of overriding (signature, return type, access)
- Where the final keyword is appropriate for a variable, method or class, and why
- How toString(), equals() and hashCode() are overridden so that two Vehicle objects with the same registration number are treated as equal

Alternative Concept and Complexity Analysis

Each subclass overrides the fee method. If a single calculateFee() with a switch on a type field were used instead of overriding, what would the solution be? Analyse the time per fee calculation for k vehicle types and the number of classes that must change when a new type is added.

Evidence Expected

Correct overriding with @Override, justified use of final, and correct Object-class method overrides.

Solution:

TASK 7 – Override and Specialise Behaviour

Campus Smart-Parking System – 4 Marks

1. Method Overriding

Method overriding allows a subclass to provide its own implementation of a method inherited from its superclass.

In the Campus Smart-Parking System, different vehicles have different parking fees. Therefore, the calculateFee() method is overridden in each vehicle subclass.

- Vehicle – declares the common fee calculation method.
- TwoWheeler – calculates the fee using the two-wheeler rate.
- Car – calculates the fee using the car rate.
- ElectricCar – calculates the fee using the electric car rate.

2. Use of @Override Annotation

The @Override annotation indicates that a subclass method overrides a superclass method.

Rules of method overriding:

1. The method name and parameter list must match.
2. The return type must be the same or a compatible covariant return type.
3. The access level cannot be more restrictive than the parent method.
4. Static methods cannot be overridden; they are hidden.
5. A final method cannot be overridden.

Example:

```
@Override
public double calculateFee(int hours) {
   return hours * 10.0;
```

}

3. Use of the final Keyword

The final keyword prevents unwanted changes.

Usage	Example	Purpose
Final variable	final String registrationNumber	Prevents changing the registration number
Final method	final void displayRules()	Prevents subclasses from overriding the method
Final class	final class ParkingRules	Prevents inheritance

In this system, the vehicle registration number is declared final because it should not change after the vehicle is created.

The calculateFee() method should not be final because each subclass needs to override it.

4. Overriding Object Class Methods

Every Java class inherits methods from the Object class.

Three important methods are:

- toString(): Returns the vehicle details as a readable string.
- equals(): Checks whether two vehicle objects have the same registration number.
- hashCode(): Returns a hash value based on the registration number.

If two vehicles have the same registration number, their equals() method must return true, and their hashCode() values must be equal.

5. Complete Java Program

File name: Main.java

Save location:

CSA0903-Java\Assignments 01\Task07\Main.java

```
import java.util.Objects;


abstract class Vehicle {
    private final String registrationNumber;
    private String ownerName;
```

```
Vehicle(String registrationNumber, String ownerName) {
    this.registrationNumber = registrationNumber;
    this.ownerName = ownerName;
}
public String getRegistrationNumber() {
    return registrationNumber;
}
public abstract double calculateFee(int hours);
@Override
public String toString() {
    return "Registration Number: " + registrationNumber
            + ", Owner: " + ownerName;
}
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (!(obj instanceof Vehicle)) {
        return false;
    }
    Vehicle other = (Vehicle) obj;
    return registrationNumber.equalsIgnoreCase(
            other.registrationNumber);
}
@Override
public int hashCode() {
    return Objects.hash(
```

```
                registrationNumber.toUpperCase());
    }
}
class TwoWheeler extends Vehicle {
    private static final double RATE = 10.0;
    TwoWheeler(String number, String owner) {
        super(number, owner);
    }
    @Override
    public double calculateFee(int hours) {
        return hours * RATE;
    }
}
class Car extends Vehicle {
    private static final double RATE = 20.0;
    Car(String number, String owner) {
        super(number, owner);
    }
    @Override
    public double calculateFee(int hours) {
        return hours * RATE;
    }
}
class ElectricCar extends Car {
    private static final double RATE = 15.0;

    ElectricCar(String number, String owner) {
        super(number, owner);
```

```
    }

    @Override
    public double calculateFee(int hours) {
        return hours * RATE;
    }
}

public class Main {
    public static void main(String[] args) {

        Vehicle v1 = new TwoWheeler("TN01AB1234", "Arun");
        Vehicle v2 = new Car("TN02CD5678", "Priya");
        Vehicle v3 = new ElectricCar("TN03EF9012", "Kumar");

        System.out.println("Vehicle Details:");
        System.out.println(v1);
        System.out.println(v2);
        System.out.println(v3);

        System.out.println("\nParking Fees for 3 Hours:");
        System.out.println("Two Wheeler: Rs." +
                v1.calculateFee(3));
        System.out.println("Car: Rs." +
                v2.calculateFee(3));
        System.out.println("Electric Car: Rs." +
                v3.calculateFee(3));
```

```
        Vehicle v4 = new Car("TN02CD5678", "Another Owner");

        System.out.println("\nEquality Checking:");
        System.out.println("Are v2 and v4 equal? " +
                v2.equals(v4));

        System.out.println("Same hash code? " +
                (v2.hashCode() == v4.hashCode()));
    }
}
```

6. Expected Output

Vehicle Details:

Registration Number: TN01AB1234, Owner: Arun

Registration Number: TN02CD5678, Owner: Priya

Registration Number: TN03EF9012, Owner: Kumar

Parking Fees for 3 Hours:

Two Wheeler: Rs.30.0

Car: Rs.60.0

Electric Car: Rs.45.0

Equality Checking:

Are v2 and v4 equal? true

Same hash code? true

7. Alternative Concept – Switch Statement vs Method Overriding

Instead of overriding calculateFee(), a single method can use a switch statement based on the vehicle type.

Example:

```
double calculateFee(String type, int hours) {
    double rate;
```

```
    switch (type) {
        case "TWO_WHEELER":
            rate = 10.0;
            break;
        case "CAR":
            rate = 20.0;
            break;
        case "ELECTRIC_CAR":
            rate = 15.0;
            break;
        default:
            rate = 0.0;
    }
    return rate * hours;
}
```

Comparison

Factor	**Method Overriding**	**Switch Statement**
Fee calculation	Each subclass calculates its own fee	One method calculates all fees
Time complexity	O(1)	O(1) for a switch with fixed cases
Adding a new vehicle	Add a new subclass	Modify the switch statement
Maintenance	Easier to extend independently	Central method needs modification
Flexibility	Supports runtime polymorphism	Depends on vehicle type value

Number of Classes to Change

- Method overriding: Only one new subclass needs to be added when introducing a new vehicle type. Existing subclasses do not need modification.
- Switch statement: The common Vehicle class or fee calculation class must be modified to include the new vehicle type. The type definition may also need updating.

Conclusion

Method overriding is suitable for the Campus Smart-Parking System because each vehicle type can calculate its parking fee independently. The @Override annotation ensures correct overriding, while final protects important data. Overriding equals(), hashCode() and toString() provides reliable vehicle comparison and readable details.

